Hello, world!

A Programming Language

Two Variables

```
x, y
```

Three Operations

```
x++
x--
x = 0 ? L1 : L2
```

Example Program

(What does it do?)

```
L1: x++
    goto
    y = 0 ? L2 : L1
L2: ...
```

The above language is "equivalent to" every PL!

But good luck writing:

- ... Quicksort
- ... Fortnite
- ... Spotify!

So Why Study Programming Languages?

Federico Fellini

A different language is a different vision of life.

So Why Study Programming Languages?

The principle of **linguistic relativity** *holds that the structure of a language affects its speakers' world view or cognition.*

Or more simply:

Programming Language shapes Programming Thought.

Language affects how ideas and computation are expressed

Course Goals

Vulcans

130 Brain

New languages come (and go ...)

There was no

- Java 30 years ago
- C# 20 years ago
- Rust 15 years ago
- WebAssembly 5 years ago

What is CSE 130 about?

- Concepts in programming languages
- Programming paradigms
- Language design and implementation

Goal: Learn the Anatomy of PL

Anatomy

- What makes a programming language?
- Which features are **fundamental** and which are **syntactic sugar**?

Goal: Learn New Languages / Constructs

Musical Score

New ways to **describe** and **organize** computation, to create programs that are:

- **Correct**
- **Readable**
- **Extendable**
- **Reusable**

Goal: How to Design new Languages

New languages being designed in industry as we speak:

- Flow, React @ Facebook
- Rust @ Mozilla
- TypeScript @ Microsoft
- Swift @ Apple
- WebAssembly @ Google + Mozilla + Microsoft

Buried in every large system is a (domain-specific) language

- DB: SQL
- Word, Excel: Formulas, Macros, VBScript
- Emacs: LISP
- Latex, shell scripts, makefiles, ...

If you work on a large system, you **will** design a new PL!

Goal: Enable You To Choose Right PL

But isn't that decided by

- Libraries
- Standards
- Hiring
- Your Boss?

Yes.

My goal: Educate tomorrow's leaders so you'll make **informed** choices.

What is CSE 130 *not* about?

Learning...

- JavaScript in January
- Haskell in February
- C++ in March

etc.

The Crew

Profs

Ranjit Jhala

- Professor at CSE since 2005
- Research: Tools and Techniques to make programs better

The Crew

Teaching Assistants

- Cole Kurashige

Tutors

- Yuzhou Xie
- Alain Zhang

Course Syllabus

Functional Programming

- Lambda calculus (2 weeks)
- Functional Programming (6 weeks)
- Building an Interpreter (3 weeks)

Logic Programming

- Prolog (~4 week, maybe, but doubtful!)

QuickSort in C

```
void sort(int arr[], int beg, int end){
  if (end > beg + 1){
    int piv = arr[beg];
    int l = beg + 1;
    int r = end;
    while (l < r){
      if(arr[l] <= piv) l++;
      else swap(&arr[l], &arr[--r]);
    if(arr[l]<=piv && arr[r-1]>piv)
      l++,r--;
    else if(arr[l]>piv && arr[r-1]<=piv)
      {l++; r--;}
    else if (arr[l]<=piv && arr[r]>piv)
      swap(&arr[l++], &arr[r--]);
    else ++l--;
    swap(&arr[--l], &arr[beg]);
    sort(arr, beg, --l);
    sort(arr, l, end);
  }
}
```

QuickSort in Haskell

```
sort []     = []
sort (x:xs) = sort ls ++ [x] ++ sort rs
  where
    ls      = [ l | l <- xs, l <= x ]
    rs      = [ r | r <- xs, x < r ]
```

(**Note:** not a wholly fair comparison...)

Course Logistics

webpage

- Calendar
- Lecture notes
- Programming assignments

piazza

- Go-to-place if you have a question or need help

Policies

- No podcasting.
- No screens (phones, laptops) in lecture.
- Yes attendance & class participation (worksheets).
- Yes exams must be done at allotted time and location.

Grading

- 10% Class participation (*worksheets*)
- 30% Programming Assignments (*codespaces*)
- 30% Two Midterm **Tu 1/27** and **Thu 2/19** (*in lecture*)
- 30% Final **Tu 3/18**
- 05% Piazza Extra Credit (*top 5* best participants)

Class participation (10%)

- "In class" worksheets handed out each lecture
- Handed in at the end of the lecture
- Turn in 75% of the worksheets to get full credit
- Responses will be graded on participation (not correctness)

Assignments (30%)

6 programming assignments

- Released online
- At least a week before due date
- Via github classroom + codespaces

Six late days

- used as **whole unit**
- 5 mins late = 1 late day
- at most 4 per assignment

No Groups

- Assignments must be submitted individually via github
- Ok to discuss with classmates, but solution must be your own

Exams (15 + 15 + 30%)

- Must be done at allotted time and location
- 2-sided ``cheat sheet"
- The final is cumulative

Your Resources

Discussion section

- Th 6:00-6:50pm (CENTER 212)

Office hours

- Every day, check **CANVAS calendar**

Piazza

- We answer during work hours and office hours (M-F)

No text

- Online lecture notes and links

Academic Integrity

Programming assignments: do not copy from classmates or from previous years

Exams done **alone**

- Zero Tolerance
- Offenders punished ruthlessly
- Please see academic integrity statement

Students with Disabilites

Students requesting accommodations for this course due to a disability or current functional limitation must provide a current **Authorization for Accommodation (AFA)** letter issued by the Office for Students with Disabilities (OSD) which is located in University Center 202 behind Center Hall.

Students are required to present their AFA letters to Faculty (please make arrangements to contact me privately) and to the CSE OSD Liaison Christina Rontell in advance so that accommodations may be arranged.

Lambda Calculus

Your Favorite Language

Probably has lots of features:

- Assignment (`x = x + 1`)
- Booleans, integers, characters, strings, ...
- Conditionals
- Loops
- `return`, `break`, `continue`
- Functions
- Recursion
- References / pointers
- Objects and classes
- Inheritance
- ...

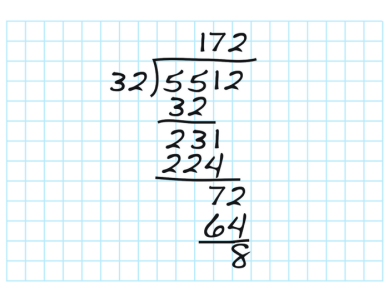
Which ones can we do without?

What is the **smallest universal language**?

What is computable?

Before 1930s

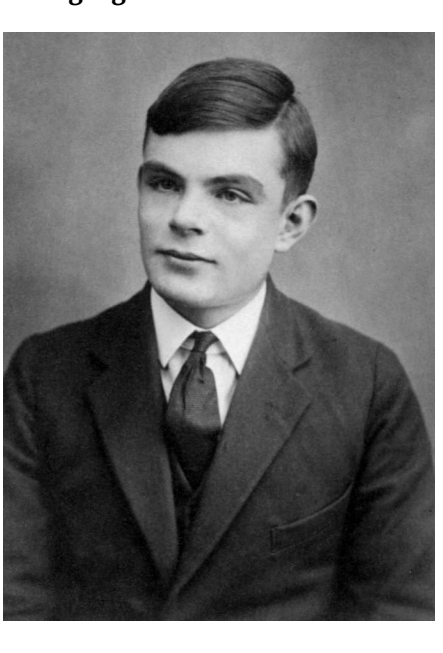
Informal notion of an **effectively calculable** function:



can be computed by a human with pen and paper, following an algorithm

1936: Formalization

What is the **smallest universal language**?



Alan Turing

Alonzo Church

The Next 700 Languages

Peter Landin

> *Whatever the next 700 languages turn out to be, they will surely be variants of lambda calculus.*

Peter Landin, 1966

The Lambda Calculus

Has one feature:

- Functions

No, *really*

- Assignment (`x = x + 1`)
- Booleans, integers, characters, strings, ...
- Conditionals
- Loops
- `return`, `break`, `continue`
- Functions
- Recursion
- References / pointers
- Objects and classes
- Inheritance
- Reflection

More precisely, *only thing* you can do is:

- **Define** a function
- **Call** a function

Describing a Programming Language

- *Syntax:* what do programs look like?
- *Semantics:* what do programs mean?
  - *Operational semantics:* how do programs execute step-by-step?

Syntax: What Programs Look Like

```
e ::= x              -- variable 'x'
    | (\x -> e)      -- function that takes a parameter 'x' and returns 'e'
    | (e1 e2)        -- call (function) 'e1' with argument 'e2'
```

Programs are **expressions** `e` (also called **λ-terms**) of one of three kinds:

- **Variable**
  - `x`, `y`, `z`
- **Abstraction** (aka *nameless* function definition)
  - `(\x -> e)`
  - `x` is the *formal* parameter, `e` is the *body*
  - "for any `x` compute `e`"
- **Application** (aka function call)
  - `(e1 e2)`
  - `e1` is the *function*, `e2` is the *argument*
  - in your favorite language: `e1(e2)`

(Here each of `e`, `e1`, `e2` can itself be a variable, abstraction, or application)

Examples

```
(\x -> x)             -- The identity function (id) that returns its input

(\x -> (\y -> y))     -- A function that returns (id)

(\f -> (f (\x -> x))) -- A function that applies its argument to id
```

QUIZ

Which of the following terms are syntactically incorrect?

A. `((\x -> x) -> y)`

B. `(\x -> (x x))`

C. `(\x -> (x (y x)))`

D. A and C

E. all of the above

Examples

```
(\x -> x)             -- The identity function (id) that returns its input

(\x -> (\y -> y))     -- A function that returns (id)

(\f -> (f (\x -> x))) -- A function that applies its argument to id
```

How do I define a function with two arguments?

- e.g. a function that takes `x` and `y` and returns `y`?

```
(\x -> (\y -> y))    -- A function that returns the identity function
                     -- OR: a function that takes two arguments
                     -- and returns the second one!
```

How do I apply a function to two arguments?

- e.g. apply `(\x -> (\y -> y))` to `apple` and `banana`?

```
(((\x -> (\y -> y)) apple) banana) -- first apply to apple,
                                   -- then apply the result to banana
```

Syntactic Sugar

instead of	we write
`\x -> (\y -> (\z -> e))`	`\x -> \y -> \z -> e`
`\x -> \y -> \z -> e`	`\x y z -> e`
`(((e1 e2) e3) e4)`	`e1 e2 e3 e4`

```
\x y -> y      -- A function that that takes two arguments
               -- and returns the second one...
```

```
(\x y -> y) apple banana -- ... applied to two arguments
```

Semantics : What Programs Mean

How do I "run" / "execute" a λ-term?

Think of middle-school algebra:

```
    (1 + 2) * ((3 * 8) - 2)
==
    3       * ((3 * 8) - 2)
==
    3       * ( 24     - 2)
==
    3       *   22
==
    66
```

Execute = rewrite step-by-step

- Following simple *rules*
- until no more rules *apply*

Rewrite Rules of Lambda Calculus

1. β-step (aka *function call*)
2. α-step (aka *renaming formals*)

But first we have to talk about **scope**

Semantics: Scope of a Variable

The part of a program where a **variable is visible**

In the expression `(\x -> e)`

- `x` is the newly introduced variable
- `e` is **the scope** of `x`
- any occurrence of `x` in `(\x -> e)` is **bound** (by the **binder** `\x`)

For example, `x` is bound in:

```
(\x -> x)

(\x -> (\y -> x))
```

An occurrence of `x` in `e` is **free** if it's *not bound* by an enclosing abstraction

For example, `x` is free in:

```
(x y)                 -- no binders at all!

(\y -> (x y))         -- no \x binder

((\x -> (\y -> y)) x) -- x is outside the scope of the \x binder;
                      -- intuition: it's not "the same" x
```

QUIZ

Is `x` *bound* or *free* in the expression `((\x -> x) x)`?

A. first occurrence is bound, second is bound

B. first occurrence is bound, second is free

C. first occurrence is free, second is bound

D. first occurrence is free, second is free

EXERCISE: Free Variables

An variable `x` is **free** in `e` if *there exists* a free occurrence of `x` in `e`

We can formally define the set of *all free variables* in a term like so:

```
FV(x)       = ???
FV(\x -> e) = ???
FV(e1 e2)   = ???
```

Closed Expressions

If `e` has *no free variables* it is said to be **closed**

- Closed expressions are also called **combinators**

What is the shortest closed expression?

Rewrite Rules of Lambda Calculus

1. β-step (aka *function call*)
2. α-step (aka *renaming formals*)

Semantics: Redex

A **redex** is a term of the form

```
((\x -> e1) e2)
```

A *function* `(\x -> e1)`

- `x` is the *parameter*
- `e1` is the *returned* expression

Applied to an argument `e2`

- `e2` is the *argument*

Semantics: β-Reduction

A **redex** b-steps to another term ...

```
(\x -> e1) e2   =b>   e1[x := e2]
```

where `e1[x := e2]` means

"`e1` with all *free* occurrences of `x` replaced with `e2`"

Computation by *search-and-replace*:

If you see an *abstraction* applied to an *argument*,

- In the *body* of the abstraction
- Replace all *free occurrences* of the *formal* by that *argument*

We say that `(\x -> e1) e2` β-steps to `e1[x := e2]`

Redex Examples

```
((\x -> x) apple)

=b> apple
```

Is this right? Ask Elsa

QUIZ

```
((\x -> (\y -> y)) apple)

=b> ???
```

A. `apple`

B. `\y -> apple`

C. `\x -> apple`

D. `\y -> y`

E. `\x -> y`

QUIZ

```
(\x -> (((y x) y) x)) apple
=b> ???
```

A. `(((apple apple) apple) apple)`

B. `(((y apple) y) apple)`

C. `(((y y) y) y)`

D. `apple`

QUIZ

```
((\x -> (x (\x -> x))) apple)

=b> ???
```

A. `(apple (\x -> x))`

B. `(apple (\apple -> apple))`

C. `(apple (\x -> apple))`

D. `apple`

E. `(\x -> x)`

EXERCISE

What is a λ-term `fill_this_in` such that

```
fill_this_in apple
=b> banana
```

ELSA: https://elsa.goto.ucsd.edu/index.html

Click here to try this exercise

A Tricky One

```
((\x -> (\y -> x)) y)

=b> \y -> y
```

Is this right?

Something is Fishy

```
(\x -> (\y -> x)) y

=b> (\y -> y)
```

Is this right?

Problem: The *free* `y` in the argument has been **captured** by `\y` in *body*!

Solution: Ensure that *formals* in the body are **different from** *free-variables* of argument!

Capture-Avoiding Substitution

We have to fix our definition of β-reduction:

```
 (\x -> e1) e2   =b>   e1[x := e2]
```

where `e1[x := e2]` means ~~" e1 with all *free* occurrences of x replaced with e2 "~~

- `e1` with all *free* occurrences of `x` replaced with `e2`
- **as long as** no free variables of `e2` get captured

Formally:

```
x[x := e]            = e

y[x := e]            = y                  -- as x /= y

(e1 e2)[x := e]      = (e1[x := e]) (e2[x := e])

(\x -> e1)[x := e]   = (\x -> e1)         -- Q: Why leave `e1` unchanged?

(\y -> e1)[x := e]
  | not (y in FV(e)) = \y -> e1[x := e]
```

Oops, but what to do if `y` is in the *free-variables* of `e`?

- i.e. if `\y -> ...` may *capture* those free variables?

Rewrite Rules of Lambda Calculus

1. β-step (aka *function call*)
2. α-step (aka *renaming formals*)

Semantics: α-Renaming

```
\x -> e   =a>   \y -> e[x := y]
  where not (y in FV(e))
```

- We rename a formal parameter `x` to `y`
- By replace all occurrences of `x` in the body with `y`
- We say that `\x -> e` α-steps to `\y -> e[x := y]`

Example:

```
(\x -> x)   =a>   (\y -> y)   =a>    (\z -> z)
```

All these expressions are **α-equivalent**

What's wrong with these?

```
-- (A)
(\f -> (f x))    =a>   (\x -> (x x))

-- (B)
((\x -> (\y -> y)) y)   =a>   ((\x -> (\z -> z)) z)
```

Tricky Example Revisited

```
((\x -> (\y -> x)) y)
                             -- rename 'y' to 'z' to avoid capture
=a> ((\x -> (\z -> x)) y)
                             -- now do b-step without capture!
=b> (\z -> y)
```

To avoid getting confused,

- you can **always rename** formals,
- so different **variables** have different **names**!

Normal Forms

Recall **redex** is a λ-term of the form

```
((\x -> e1) e2)
```

A λ-term is in **normal form** if it *contains no redexes*.

QUIZ

Which of the following term are **not** in *normal form* ?

A. `x`

B. `(x y)`

C. `((\x -> x) y)`

D. `(x (\y -> y))`

E. C and D

Semantics: Evaluation

A λ-term `e` **evaluates to** `e'` if

1. There is a sequence of steps

```
e =?> e_1 =?> ... =?> e_N =?> e'
```

where each `=?>` is either `=a>` or `=b>` and `N >= 0`

2. `e'` is in *normal form*

Examples of Evaluation

```
((\x -> x) apple)

  =b> apple
```

```
(\f -> f (\x -> x)) (\x -> x)

  =?> ???
```

```
(\x -> x x) (\x -> x)
  =?> ???
```

Elsa shortcuts

Named λ-terms:

```
let ID = (\x -> x)  -- abbreviation for (\x -> x)
```

To substitute name with its definition, use a `=d>` step:

```
(ID apple)
  =d> ((\x -> x) apple)  -- expand definition
  =b> apple              -- beta-reduce
```

Evaluation:

- `e1 =*> e2`: `e1` reduces to `e2` in 0 or more steps
  - where each step is `=a>`, `=b>`, or `=d>`
- `e1 =~> e2`: `e1` evaluates to `e2` and `e2` is **in normal form**

EXERCISE

Fill in the definitions of `FIRST`, `SECOND` and `THIRD` such that you get the following behavior in `elsa`

```
let FIRST  = fill_this_in
let SECOND = fill_this_in
let THIRD  = fill_this_in

eval ex1 :
  FIRST apple banana orange
  =*> apple

eval ex2 :
  SECOND apple banana orange
  =*> banana

eval ex3 :
  THIRD apple banana orange
  =*> orange
```

ELSA: https://elsa.goto.ucsd.edu/index.html

Click here to try this exercise

Non-Terminating Evaluation

```
((\x -> (x x)) (\x -> (x x)))

  =b> ((\x -> (x x)) (\x -> (x x)))
```

Some programs loop back to themselves ... *never* reduce to a normal form!

This combinator is called Ω

What if we pass Ω as an argument to another function?

```
let OMEGA = ((\x -> (x x)) (\x -> (x x)))

((\x -> (\y -> y)) OMEGA)
```

Does this reduce to a normal form? Try it at home!

Programming in λ-calculus

Real languages have lots of features

- Booleans
- Records (structs, tuples)
- Numbers
- Lists
- **Functions** [we got those]
- Recursion

Lets see how to *encode* all of these features with the λ-calculus.

Syntactic Sugar

instead of	we write
`\x -> (\y -> (\z -> e))`	`\x -> \y -> \z -> e`
`\x -> \y -> \z -> e`	`\x y z -> e`
`(((e1 e2) e3) e4)`	`e1 e2 e3 e4`

```
\x y -> y      -- A function that that takes two arguments
               -- and returns the second one...

(\x y -> y) apple banana -- ... applied to two arguments
```

```
-- ...
```

QUIZ: Church Numerals

Which of these is a valid encoding of `ZERO` ?

- **A** `let ZERO = \f -> n`
- **B** `let ZERO = \f -> f`
- **C** `let ZERO = \f -> f x`
- **D** `let ZERO = \x -> x`
- **E** None of the above

Does this function look familiar?

λ-calculus: Increment

```
-- Call `f` on `n` one more time than `n` does
let INC  = \n -> (\f x -> ???)
```

Example:

```
eval: inc_zero :
  INC ZERO
=b> (\n f x -> f (n f x)) ZERO
=b> \f x -> f (ZERO f x)
=*> \f x -> f x
=b> ONE
```

EXERCISE

Fill in the implementation of `ADD` so that you get the following behavior

Click here to try this exercise

```
let ZERO = \f x -> x
let ONE  = \f x -> f x
let TWO  = \f x -> f (f x)
let INC  = \n f x -> f (n f x)

let ADD  = fill_this_in

eval: add_zero_zero:
  ADD ZERO ZERO ==> ZERO

eval: add_zero_one:
  ADD ZERO ONE ==> ONE

eval: add_zero_two:
  ADD ZERO TWO ==> TWO

eval: add_one_zero:
  ADD ONE ZERO ==> ONE

eval: add_one_one:
  ADD ONE ONE ==> TWO

eval: add_two_zero:
  ADD TWO ZERO ==> TWO
```

QUIZ

How shall we implement `ADD` ?

A. `let ADD = \n m -> n INC m`

B. `let ADD = \n m -> INC n m`

C. `let ADD = \n m -> n m INC`

D. `let ADD = \n m -> n (m INC)`

E. `let ADD = \n m -> n (INC m)`

λ-calculus: Addition

```
-- Call `f` an `x` exactly `n + m` times
let ADD = \n m -> n INC m
```

Example:

```
eval: add_one_zero :
  ADD ONE ZERO
  ==> ONE
```

QUIZ

How shall we implement `MULT` ?

A. `let MULT = \n m -> n ADD m`

B. `let MULT = \n m -> n (ADD m) ZERO`

C. `let MULT = \n m -> m (ADD n) ZERO`

D. `let MULT = \n m -> n (ADD m) ZERO`

E. `let MULT = \n m -> (n ADD m) ZERO`

λ-calculus: Multiplication

```
-- Call `f` an `n * m` times
let MULT = \n m -> n (ADD m) ZERO
```

Example:

```
eval: two_times_three :
  MULT TWO ONE
  ==> TWO
```

Programming in λ-calculus

- **Booleans** (done)
- **Records** (structs, tuples) (done)
- **Numbers** (done)
- **Lists**
- **Functions** (we got those)
- Recursion

λ-calculus: Lists

Lets define an API to build lists in the *λ*-calculus.

An Empty List

```
NIL
```

Constructing a list

A list with 4 elements.

```
CONS apple (CONS banana (CONS cantaloupe (CONS dragon NIL)))
```

Intuitively `CONS h t` creates a new list with

- head `h`
- tail `t`

Destructing a list

- `HEAD l` returns the *first* element of the list
- `TAIL l` returns the *rest* of the list

```
HEAD (CONS apple (CONS banana (CONS cantaloupe (CONS dragon NIL))))
==> apple

TAIL (CONS apple (CONS banana (CONS cantaloupe (CONS dragon NIL))))
==> CONS banana (CONS cantaloupe (CONS dragon NIL))
```

λ-calculus: Lists

```
let NIL  = ???
let CONS = ???
let HEAD = ???
let TAIL = ???

eval: eval1:
  HEAD (CONS apple (CONS banana (CONS cantaloupe (CONS dragon NIL))))
  ==> apple

eval: eval1:
  TAIL (CONS apple (CONS banana (CONS cantaloupe (CONS dragon NIL))))
  ==> CONS banana (CONS cantaloupe (CONS dragon NIL))
```

EXERCISE: Nth

Write an implementation of `GetNth` such that

- `GetNth n l` returns the n-th element of the list `l`.

Assume that `l` has *n or more elements*

```
let GetNth = ???

eval: nth1 :
  GetNth ZERO (CONS apple (CONS banana (CONS cantaloupe NIL)))
  ==> apple

eval: nth1 :
  GetNth ONE (CONS apple (CONS banana (CONS cantaloupe NIL)))
  ==> banana

eval: nth2 :
  GetNth TWO (CONS apple (CONS banana (CONS cantaloupe NIL)))
  ==> cantaloupe
```

Click here to try this in sbas

λ-calculus: Recursion

I want to write a function that sums up natural numbers up to `n`:

```
let SUM = \n -> ... 0 + 1 + 2 + ... + n
```

such that we get the following behavior

```
eval: evlsm0: SUM ZERO ==> ZERO
eval: evlsm1: SUM ONE ==> ONE
eval: evlsm2: SUM TWO ==> THREE
eval: evlsm3: SUM THREE ==> SIX
```

Can we write sum using **Church Numerals**?

Click here to try this in Sbas

QUIZ

You can write `SUM` using numerals but its tedious.

Is this a correct implementation of `SUM`?

```
let SUM = \n -> ITE (ISZ n)
                    ZERO
                    (ADD n (SUM (DEC n)))
```

A. Yes

B. No

Ha!

- Named terms in Sbas are just syntactic sugar
- To translate an Sbas term to a λ-calculus replace each name with its definition

```
\n -> ITE (ISZ n)
          ZERO
          (ADD n (SUM (DEC n)))  -- But SUM is not yet defined!
```

Recursion:

- Inside *this* function
- Want to call the *same* function on `DEC n`

Looks like we can't do recursion!

- Requires being able to refer to functions *by name*,
- But λ-calculus functions are *anonymous*.

Right?

λ-calculus: Recursion

Think again!

Recursion:

Instead of

- ~~Inside *this* function I want to call the *same* function on `DEC n`~~

Lets try

- Inside *this* function I want to call *some* function `rec` on `DEC n`
- And BTW, I want `rec` to be the *same* function

Step 1: Pass in the function to call "recursively"

```
let STEP =
  \rec -> \n -> ITE (ISZ n)
                    ZERO
                    (ADD n (rec (DEC n))) -- Call some rec
```

Step 2: Do some magic to `STEP`, so `rec` is itself

```
\n -> ITE (ISZ n) ZERO (ADD n (rec (DEC n)))
```

That is, obtain a term `MAGIC` such that

```
MAGIC =*> STEP MAGIC
```

λ-calculus: Fixpoint Combinator

Wanted: a λ-term `FIX` such that

- `FIX STEP` calls `STEP` with `FIX STEP` as the first argument:

```
(FIX STEP) =*> STEP (FIX STEP)
```

(In math: a *fixpoint* of a function $f(x)$ is a point x, such that $f(x) = x$)

Once we have it, we can define:

```
let SUM = FIX STEP
```

Then by property of `FIX` we have:

```
SUM   =*>   FIX STEP  =*>   STEP (FIX STEP)   =*>   STEP SUM
```

and so now we compute:

```
eval sum_two:
  SUM TWO
  =*> STEP SUM TWO
  =*> ITE (ISZ TWO) ZERO (ADD TWO (SUM (DEC TWO)))
  =*> ADD TWO (SUM (DEC TWO))
  =*> ADD TWO (SUM ONE)
  =*> ADD TWO (STEP SUM ONE)
  =*> ADD TWO (ITE (ISZ ONE) ZERO (ADD ONE (SUM (DEC ONE))))
  =*> ADD TWO (ADD ONE (SUM (DEC ONE)))
  =*> ADD TWO (ADD ONE (SUM ZERO))
  =*> ADD TWO (ADD ONE (ITE (ISZ ZERO) ZERO (ADD ZERO (SUM DEC ZERO))))
  =*> ADD TWO (ADD ONE (ZERO))
  =*> THREE
```

How should we define `FIX` ???

The Y combinator

Remember Ω?

```
(\x -> x x) (\x -> x x)
=b> (\x -> x x) (\x -> x x)
```

This is *self-replcating code*! We need something like this but a bit more involved...

The Y combinator discovered by Haskell Curry:

```
let FIX   = \stp -> (\x -> stp (x x)) (\x -> stp (x x))
```

How does it work?

```
eval fix_step:
  FIX STEP
  =d> (\stp -> (\x -> stp (x x)) (\x -> stp (x x))) STEP
  =b> (\x -> STEP (x x)) (\x -> STEP (x x))
  =b> STEP ((\x -> STEP (x x)) (\x -> STEP (x x)))
  --       ^^^^^^^^^^ this is FIX STEP ^^^^^^^^^^
```

That's all folks, Haskell Curry was very clever.

Next week: We'll look at the language named after him (`Haskell`)
